Technical Journal Entry: Resource Storage & Seed Bank

- **Module:** Resource Storage and Seed Bank Initialization (Plant class + UI unlock)
- **Time/Space Complexity:** O(1) for Plant object instantiation; O(k) time/space for populating the seed dropdown, where k is the number of plant tuples passed to `addPlantsToDropdown`.
- **Architectural Choice:** Used a simple class (`Plant`) to model garden resource entities with encapsulated state (`name`, `water_need`, `health`), then called two bridge functions (`updateResources`, `addPlantsToDropdown`) to sync Python state with the JS/DOM layer and unlock the Resources panel.
- **AI Reflection Note:**
    - **The Prompt Used (RTCC):**
      "Act as an expert Python developer. Task: Write a Plant class and initialize the garden's resource system and seed dropdown. Context: Python 3.11+, vanilla Python compatible with browser Pyodide execution, must call exact JS bridge functions `updateResources` and `addPlantsToDropdown`. Constraints: Must include a `__init__` storing name, water_need, and health=100, print a creation message, create at least one Plant object, call `updateResources` with exact specified values, call `addPlantsToDropdown` with at least 3 tuples, and end with an exact success print statement."
    - **What the AI Initially Got Wrong:**
      N/A — output matched requirements on first generation (no missing fields, correct function signatures, correct tuple format).
    - **How I Fixed It:**
      N/A — no correction cycle was needed for this module.
- **Screenshots:** *[insert screenshot of unlocked Resources panel + console output here]*

Technical Prompt Log

Module: Sanctuary Garden Grid (2D Array) — Topic 4 Implementation

Session Context: Diagnosing and completing the garden grid feature in the Python-Based Garden Simulator (DSA finals project), from initial bug discovery through full implementation.

Entry 1 — Diagnosing the Locked Grid

- **Prompt Used:** "clicking on the interactive summary reveals a blurred grid. what is this?"
- **What the AI Found:** The `view-game` section carries a CSS `.locked` class (blur + disabled pointer events) that only clears when `unlockFeature("grid")` runs, which is gated behind a Topic 4 code submission — but `topicTemplates` only defined Topics 1–3, so no UI path existed to trigger it.
- **Outcome:** Root cause identified; no fix applied yet.

Entry 2 — Requesting Workarounds

- **Prompt Used:** "give me workarounds to unlock this. since there is no 4th topic."
- **What the AI Provided:** Four options — console call to `unlockFeature("grid")`, running the same call from the Python editor (since it's registered as a Pyodide global), a `javascript:` bookmarklet, and the proper fix of adding a real Topic 4 entry.
- **Outcome:** User applied the console/editor workaround to confirm the unlock mechanism worked.

Entry 3 — Surfacing the Deeper Bug

- **Prompt Used:** "did this and it unlocked but it's not functionable"
- **What the AI Initially Got Wrong (pre-existing code, not AI-introduced):** `renderGarden()` never attached any `onclick` handler to grid cells, and `gardenPlants` was a flat object indexed by a single position rather than a 2D structure — so unlocking only removed the visual overlay, not the missing interactivity.
- **Outcome:** Problem scoped as two separate issues: (1) no Topic 4 exists, (2) grid has no click logic or real 2D structure.

Entry 4 — Full Implementation (RTCC)

- **Prompt Used:** "'2D list and array' is a requirement for this project... follow the structure and function of the 3 base topics... create a 4th topic and make the existing grid

functioning properly... do not introduce unauthorized external libraries... [list: Tailwind CSS, Lucide Icons, Pyodide, Vanilla JS/HTML5 DOM]... respond in a code snippet that highlights every single change."

- **What the AI Initially Got Wrong:** N/A on this pass — the AI directly refactored `gardenPlants` into a true `[row][col]` 2D array, added `initGardenGrid(rows, cols)`, rewrote `addPlantToGrid(row, col, name)`, added real click handlers in `renderGarden()`, gated the Topic 4 unlock on `code.includes("initGardenGrid")` (matching Topics 1–3's pattern), and added the Topic 4 template using a Python 2D list comprehension.
- **How It Was Verified:** Each change was isolated into its own labeled diff (function/section named as its "location" in source), followed by the complete revised file delivered as a downloadable artifact.

Entry 5 — Requesting Student-Facing Input Code

- **Prompt Used:** "generate input code to put in the student code editor for topic 4"
- **What the AI Provided:** The exact starter snippet (`garden_grid = [[None for _ in range(cols)] for _ in range(rows)]` → `initGardenGrid(rows, cols)`) formatted for direct paste into the editor, with a short explanation of why it satisfies both the 2D-array requirement and the unlock condition.

Entry 6 — Formalizing the Reusable Prompt Pattern

- **Prompt Used:** "generate prompt to receive this exact format of code snippets and the changes made. add a location on where the source code is the changes made. after that, an additional file for the revised version."
- **Outcome:** A reusable prompt template was produced, codifying: numbered per-change diff blocks with named source locations, followed by a separately delivered full revised file rather than inline pasting.

Architectural Summary: The Topic 4 fix mirrors the existing Topics 1–3 pattern exactly — student writes code in the editor, runs it, `runPythonCode()` checks for a required function call in the submitted code, and only then unlocks the UI panel via `unlockFeature()`. The grid's `gardenPlants` array is now a genuine 2D list (`rows × cols`), satisfying the DSA 2D-array requirement on both the Python and JavaScript sides, with click handlers wired directly to `(row, col)` indices rather than a flat offset.

Technical Journal Entry:

- **Module:** Skill Tree (Strict Binary Search Tree with BFS, DFS and Node Mutation)
- **Time/Space Complexity:**
    - BST insert and search: O(h) time, which is O(log n) for a balanced tree and O(n) worst case.
    - BFS layout and tier scaling: O(n) time, O(w) extra space for the queue, where w is the widest level.
    - DFS cascading refund and branch value: O(n) time, O(h) space for the recursion or stack.
    - Node mutation (buy or refund of one node): O(1).
    - O(n) space overall for storing n skill nodes.
- **Architectural Choice:** I used a strict Binary Search Tree where every skill has a unique `Skill_ID`, a smaller ID goes left and a larger ID goes right. Each skill has exactly one prerequisite, its parent, so converging dependencies are not possible. BFS (a queue) walks the tree level by level to assign each node a row, a column and a depth tier, and the tier drives the cost formula `Base Cost × 1.5^tier`. DFS (post-order) refunds a skill by first refunding everything in its left and right subtrees, so no unlocked skill is ever left without its parent. A stack-based DFS also sums the coins and hope invested in a branch. Buying or refunding mutates the node in place (`is_unlocked`, `current_rank`). That triggers a recalculation of global stats such as water capacity and harvest yield, and the UI redraws the node.
- **AI Reflection Note:**
    - **The Prompt Used (RTCC):**
      "Act as an expert Python and JavaScript developer. Task: Implement a skill tree for my garden simulator using a strict Binary Search Tree. Use BFS for level-order layout (row and column) and tier cost scaling (Base Cost × 1.5^tier). Use DFS for cascading refunds and branch value calculation. Use node mutation (`is_unlocked`, `current_rank`) to update game stats and the UI. Context: single HTML file (the provided Week 3 code), Python 3.11+ vanilla Python running in browser Pyodide, existing Topics 1–5 must keep working. Constraints: no converging dependencies, a maximum of two children per node, each skill has one parent prerequisite, and nothing crashes on invalid actions."
    - **What the AI Initially Got Wrong:**
        1. **Layout:** The first version of the tree UI clipped the leftmost and rightmost leaf nodes at the edge of the panel, and the connecting lines ran through the skill name labels.
        2. **Code injection:** The first attempt to inject the Python code into the JavaScript template literal failed because a stray backtick in a Python docstring would have broken the page. A safety check caught it before it shipped.
        3. **Tests:** Two failing checks in the first browser test run came from the test itself, not the game. One was a case-sensitive text match against

uppercase CSS text, and the other was a console error from a blocked font request.

- **How I Fixed It:**
    1. I added padding with an inner container so edge nodes fit, gave labels a solid background so lines pass behind them, and tuned node sizes so the tree fits without horizontal scrolling on desktop.
    2. I removed the backtick from the docstring and added an assertion that rejects backticks, backslashes and `${` in the injected code.
    3. I made the text match case-insensitive and ignored the blocked-resource console message. The test run then passed 38/38 checks in a real headless browser with Pyodide.

- **Screenshots:**

Python-Based Garden Simulator

SDG 2, 13, 15

DAY 1 | SCORE 0%

Resource Storage

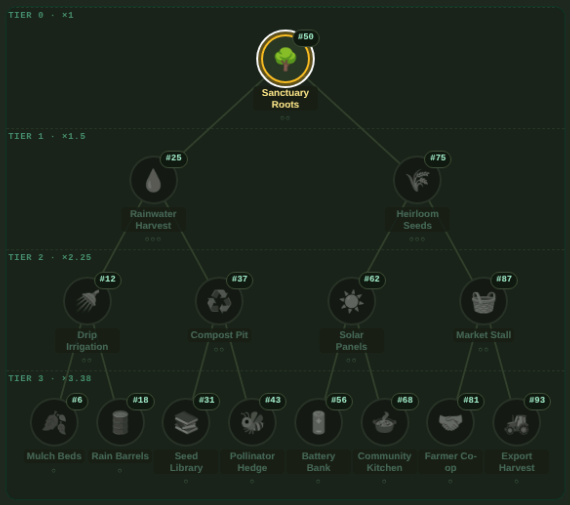
- Water Reserves: 200 / 200 L
- Seed Inventory: 90 / 100
- Solar Energy: 110 / 120 Wh
- Community Hope: 80 / 100
- Coins / Dues: 999 Coins

Interactive Sanctuary | Skill Tree | Student Code Editor

Sanctuary Skill Tree

Strict BST: **left = smaller Skill_ID**, **right = larger**. One prerequisite per skill (its parent).

COINS 999 | UNLOCKED 0 / 15



BFS level order: #50 → #25 → #75 → #12 → #37 → #62 → #87 → #6 → #18 → #31 → #43 → #56 → #68 → #81 → #93

Last DFS refund (post-order): no refund yet

Sanctuary Roots — Skill_ID 50

+10 Seed capacity per rank

AVAILABLE | Rank 0 / 2

Prerequisite: **none (root)**

Children: **◀ #25 Rainwater Harvest · ▶ #75 Heirloom Seeds**

BFS tier cost: **8 × 1.5⁰ = 8 coins / rank**

DFS branch value: **0 coins in 0 unlocked skills**

Unlock · 8 coins | Nothing to refund

Refunding a skill makes DFS dive into its left and right subtrees and refund every unlocked descendant bottom-up.

◀ LEFT BRANCH OF ROOT
#25 Rainwater Harvest
0 coins +0 hope
0 unlocked skills · summed by DFS

▶ RIGHT BRANCH OF ROOT
#75 Heirloom Seeds
0 coins +0 hope
0 unlocked skills · summed by DFS

GLOBAL STATS (RECALCULATED AFTER EVERY NODE MUTATION)

- 💧 Water capacity: 200 → 200 L — no bonus yet
- 🌱 Seed capacity: 100 → 100 — no bonus yet
- ☀️ Solar capacity: 120 → 120 Wh — no bonus yet
- 💗 Community Hope: +0 — bar 80 / 100
- 🌾 Harvest yield: +0% — harvest = 5 coins

Climate Queue (FIFO)

- Typhoon
- Drought

Action History (Stack)

- → Skill Tree Initialized (BST + BFS + DFS)
- → Growth Stage Hierarchy Initialized (Tree)
- → Sanctuary Garden Grid Initialized (2D Array)
- → planted at [1] [1]
- → planted at [1] [2]
- → Resources & Seed Bag Initialized

Skill Tree unlocked: open the Skill Tree tab

Undo (Stack Pop)

Growth Hierarchy (Tree)

- 🌰 ↓ child node
- 🌱 ↓ child node
- 🌳 ↻ loops back to root

Click a planted tile to cycle it through the tree.

Python-Based Garden Simulator

SDG 2, 13, 15

DAY 1 | SCORE 0%

Resource Storage

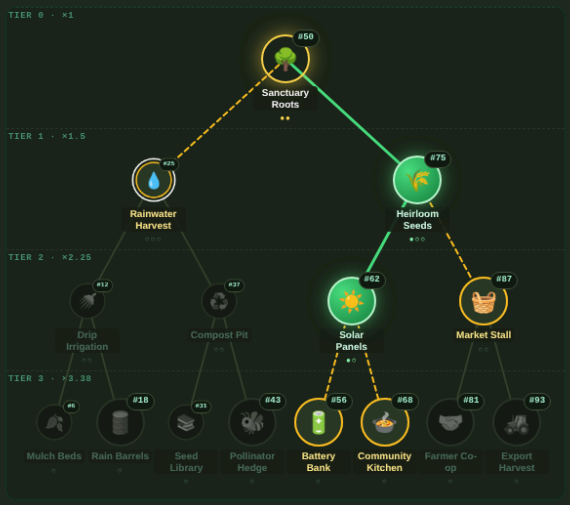
- Water Reserves: 200 / 200 L
- Seed Inventory: 90 / 120
- Solar Energy: 110 / 140 Wh
- Community Hope: 80 / 100
- Coins / Dues: 935 Coins

Interactive Sanctuary | Skill Tree | Student Code Editor

Sanctuary Skill Tree

Strict BST: **left = smaller Skill_ID**, **right = larger**. One prerequisite per skill (its parent).

COINS 935 | UNLOCKED 3 / 15



BFS level order: #50 → #25 → #75 → #12 → #37 → #62 → #87 → #6 → #18 → #31 → #43 → #56 → #68 → #81 → #93

Last DFS refund (post-order): #6 → #12 → #31 → #37 → #25

Rainwater Harvest — Skill_ID 25

+25 L Water capacity per rank

AVAILABLE

Rank 0 / 3

Prerequisite: **#50 Sanctuary Roots ✓**

Children: **◀ #12 Drip Irrigation · ▶ #37 C… Pit**

BFS tier cost: **8 × 1.5¹ = 12 coins / rank**

DFS branch value: **0 coins in 0 unlocked skills**

Unlock · 12 coins | Nothing to refund

Refunding a skill makes DFS dive into its left and right subtrees and refund every unlocked descendant bottom-up.

◀ LEFT BRANCH OF ROOT
#25 Rainwater Harvest
0 coins +0 hope
0 unlocked skills · summed by DFS

▶ RIGHT BRANCH OF ROOT
#75 Heirloom Seeds
48 coins +0 hope
2 unlocked skills · summed by DFS

GLOBAL STATS (RECALCULATED AFTER EVERY NODE MUTATION)

- Water capacity: 200 → 200 L — no bonus yet
- Seed capacity: 100 → 120 — +20 from skills
- Solar capacity: 120 → 140 Wh — +20 Wh from skills
- Community Hope: +0 — bar 80 / 100
- Harvest yield: +10% — harvest = 6 coins

Climate Queue (FIFO)

- Typhoon
- Drought

Action History (Stack)

- → Refunded Rainwater Harvest branch: 5 skill(s), +143 coins
- → Unlocked Solar Panels (rank 1/2) for 27 coins
- → Unlocked Heirloom Seeds (rank 1/3) for 21 coins
- → Unlocked Seed Library (rank 1/1) for 30 coins
- → Unlocked Compost Pit (rank 1/2) for 27 coins
- → Unlocked Mulch Beds (rank 1/1) for 27 coins
- → Unlocked Drip Irrigation (rank 1/2) for 23 coins
- → Unlocked Rainwater Harvest (rank 3/3) for 12 coins

Refunded Rainwater Harvest branch: 5 skill(s), +143 coins

Undo (Stack Pop)

Growth Hierarchy (Tree)

- ↓ child node
- ↓ child node
- ↻ loops back to root

Click a planted tile to cycle it through the tree.

ASYNCHRONOUS ACTIVITY::

Sorting, Hash Tables and the Garden Simulator

Sorting Algorithms

Bubble Sort. It walks through the list comparing each pair of neighbours and swaps them when they are in the wrong order, so the largest unsorted value drifts to the end on every pass. It repeats this on the shrinking unsorted part, and it can stop early when a whole pass makes no swaps, which is why an already sorted list is its best cas.

Insertion Sort. It treats the left part of the list as already sorted, takes the next value and slides it left past every larger value until it fits, like sorting playing cards in your hand. When the list is nearly sorted each slide is short, so it runs quickly; when the list is in reverse order every value must travel the whole sorted part.

Selection Sort. Each round it scans the whole unsorted part to find the lowest value and moves it to the front of that part, so the sorted part grows by one per round. Because it always scans everything that is left, the starting order of the data barely changes how much work it does.

Quick Sort. It picks a pivot value and partitions the list so that smaller values end up on one side and larger values on the other, which puts the pivot in its final position. It then repeats the same steps on each side recursively. Balanced splits make it fast, while a bad pivot, such as the largest value every time, makes it slow.

Merge Sort. It splits the list in half again and again until every piece has one element, then merges the pieces back together by repeatedly taking the smaller front value of two sorted halves. It splits and merges the same way whatever the input order, so its running time hardly changes between inputs.

Algorithm	Best Time	Average Time	Worst Time	Extra Space	Stable?
Bubble	O(n) (sorted input, early exit)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Insertion	O(n) (sorted input)	$O(n^2)$	$O(n^2)$ (reverse order)	O(1)	Yes

Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No (usual swap version)
Quick	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	No
Merge	O(n log n)	O(n log n)	O(n log n)	O(1)	Yes

Hash Tables, Collisions and Rehashing Notes

Hash tables, hash functions and collisions

A hash table stores data in an array of buckets. A hash function turns a key into a bucket index, so searching, adding and deleting are O(1) on average, compared with O(n) for arrays and linked lists. W3Schools' example hash function adds up the ASCII codes of a name and takes the remainder after dividing by 10, which sends 'Stuart' to bucket 3. A good hash function is quick to compute, gives the same result every time for the same key, and spreads keys evenly over the buckets.

What a collision is. A collision happens when two different keys hash to the same bucket. In W3Schools' example, 'Lisa' is already stored in bucket 3 when 'Stuart' also hashes to bucket 3. Collisions cannot be avoided in general, because there are far more possible keys than buckets, so every hash table needs a plan for them.

A worked example. Take a table of 5 buckets with h(key) = key mod 5 and insert 15, 25, 7, 12. Keys 15 and 25 both map to bucket 0, and 7 and 12 both map to bucket 2.

- **Chaining:** bucket 0 holds the chain 15 -> 25 and bucket 2 holds 7 -> 12. Every key stays in the bucket its hash chose.
- **Open addressing (linear probing):** 15 takes slot 0, so 25 probes to slot 1. 7 takes slot 2, so 12 probes to slot 3. Keys end up in slots their hash did not choose.

Rehashing

What it is. Rehashing means building a new, larger bucket array and re-inserting every existing key into it. Every key has to be hashed again because its bucket index depends on the table size: with 4 buckets a key whose hash value is 13 goes to bucket 13 mod 4 = 1, but with 8 buckets it goes to bucket 13 mod 8 = 5.

Why it is triggered. The load factor is the number of stored entries divided by the number of buckets, n / m. Average bucket length under chaining equals the load factor, so as the load factor grows, chains get longer and the O(1) average no longer holds. Implementations therefore set a maximum load factor; Java's HashMap, for example, doubles its table when the load factor reaches 3/4. When an insertion would push the load factor past that limit, the table is rehashed.

How the table scales. The usual rule is to roughly double the table size. One rehash costs O(n) because all n entries are moved, but doubling makes the cost O(1) per insertion when averaged over many insertions (amortized), because rehashes get rarer as the table grows. Growing by a fixed amount, such as 100 buckets at a time, would

instead give linear amortized time, which is why the doubling matters. Rehashing also clears out lazily deleted markers when open addressing is used.

Small trace. A table starts with m = 8 and a limit of 0.75. Six entries fit (6/8 = 0.75). Inserting the 7th would make 7/8 = 0.875, so the table first grows to 16 buckets, re-inserts the 6 keys, and then adds the 7th.

Garden Simulator Project Integration (HarBEST!)

What is implemented in the game

The sorting algorithms and the hash table are implemented in the current build of my Garden Simulator, HarBEST! v3 (single HTML file, JavaScript with a Python/Pyodide side). This section is based on reading that source.

Sorting.

- `SORTS` contains Bubble, Insertion, Selection, Quick and Merge. Each takes a comparator function and counts comparisons and writes.
- `runSort(kind, arr, lt)` copies the array, sorts it and returns the sorted array with its compare count, write count and time in milliseconds.
- `sortBy(arr, key, desc)` is the hybrid: it uses Insertion Sort for 12 items or fewer and Merge Sort for longer lists.
- Where it runs: the upgrade cards (sorted by price, name or power score), the best-buy bundle, the back-to-front draw order of the plots, the Lab's largest-chains list, and the day-results rows. When Pyodide loads, the day-results ranking is done by the Python version of the same step, which uses Python's stable `list.sort`.
- Where to see it: Menu, then Lab, then the Sorting race, which times all five algorithms on 50, 200 or 800 items and checks that each result is sorted.

Hash table.

- `HashTable` is an array of buckets whose chains are linked nodes, so collisions use chaining. The hash function is FNV-1a on the key as a string, reduced with the remainder after dividing by the bucket count.
- Before each insert it checks whether (entries + 1) / buckets would exceed 0.75. If so it doubles the bucket array and re-inserts every entry, which is rehashing.
- `stats()` reports buckets, entries, load factor, collisions, rehashes, longest chain and probes. The Lab's Hash table card shows these for the owned-upgrade set and for a demo table holding all 100 upgrade IDs.
- Where it runs: `OWN` (owned upgrades) and `ACHS` (achievements) are sets built on it, and it also backs the memo caches for upgrade path costs, the knapsack planner, the Python cross-checks, and the tractor's queued-row set.

Crop inventory and leaderboard sorting

Choice of two algorithms. Basic: Insertion Sort. Advanced: Merge Sort. This is the same pairing `sortBy` already uses.

- Insertion Sort suits short lists, such as a top-10 crates panel, and lists that are already nearly sorted.
- Merge Sort stays at O(n log n) on any input and is stable, so inventory items or players with equal scores keep their earlier order.

Inventory objects. A harvested crop becomes an object such as `{ id, crop, value, weight, grow, day }`, where `grow` is the growth time in seconds.

(a) Sorting harvested crops by weight, value or growth duration. The player picks a key and the game calls the same sorting function with a different comparator:

- By value: highest first. By weight: heaviest first. By growth duration: fastest first.
- A small list (12 or fewer) is handled by Insertion Sort. A full barn of crates is handled by Merge Sort.
- To sort by two things at once, such as value and then weight, sort by weight first and then by value. Stability keeps equal-value crates in weight order.

(b) Ranking seasonal scores and leaderboards. Each finished season is stored as `{ season, farm, money, diamonds, days }`. The ranking uses Merge Sort with a tie-break comparator: higher money first, then more diamonds, then fewer days.

(c) Code showing the sort processing inventory objects.

```
// Already in HarBEST! v3: the hybrid sort
function sortBy(arr, key, desc = false) {
  const lt = desc ? (x, y) => key(x) > key(y) : (x, y) => key(x) < key(y);
  return runSort(arr.length <= 12 ? 'insertion' : 'merge', arr, lt).arr;
}

// How it processes a list of garden inventory objects
const crates = [
  { id: 1, crop: 'tomato', value: 8,  weight: 0.21, grow: 4.2 },
  { id: 2, crop: 'radish', value: 3,  weight: 0.09, grow: 2.1 },
  { id: 3, crop: 'corn',   value: 80, weight: 0.52, grow: 9.8 },
  { id: 4, crop: 'potato', value: 14, weight: 0.33, grow: 5.6 },
];
const byValue  = sortBy(crates, c => c.value, true);   // corn, potato, tomato, radish
const byWeight = sortBy(crates, c => c.weight, true);
const byGrowth = sortBy(crates, c => c.grow);          // fastest-growing first

// Leaderboard: stable merge sort with a tie-break comparator
const better = (a, b) => a.money > b.money ||
  (a.money === b.money && (a.diamonds > b.diamonds ||
  (a.diamonds === b.diamonds && a.days < b.days)));
const ranked = runSort('merge', seasons, better).arr;
```

Trace of the value sort (4 items, so Insertion Sort runs). Start: [tomato 8, radish 3, corn 80, potato 14], highest value first.

1. radish 3 is not larger than tomato 8, so it stays: [tomato 8, radish 3, corn 80, potato 14].
2. corn 80 is larger than radish 3 and tomato 8, so both shift right and corn goes first: [corn 80, tomato 8, radish 3, potato 14].
3. potato 14 is larger than radish 3 and tomato 8 but not corn 80, so it goes second: [corn 80, potato 14, tomato 8, radish 3].

Fast plant lookup and collision handling

Structure. The plant registry is a `HashTable` whose key is a plot coordinate string such as '3,4', or a unique plant ID, and whose value is a plant record: `{ uid, crop, water, waterNeeded, health, sunNeed }`.

- A plant is added to the table when it spawns and removed when it is harvested or cleared, so lookups by coordinate or ID take O(1) on average and do not scan the whole grid.
- It is used for things that are not row-and-column arithmetic: inspecting a plant on hover or tap, linking an inventory crate to the plant it came from, and looking up plant data by ID.
- The tight per-frame watering loops keep using direct array indexing, because indexing a dense square grid is cheaper than hashing.
- The game's hash table already uses FNV-1a, which accepts string keys like these.

Collision technique: separate chaining. When two plot keys land in the same bucket, they share that bucket's chain. I chose chaining for these reasons:

1. It is the technique my `HashTable` already implements and tests, so the plant registry reuses working code.
2. Plants are constantly removed (every harvest) and re-added (every respawn). Chaining deletes by unlinking a node, while open addressing needs lazy-deletion markers [11].
3. Chaining still works if the load factor goes above 1 [11], so a burst of new plants during an expansion cannot make inserts fail.
4. Open addressing has real advantages (no list pointers, tighter memory use) [17], but at 25 to 81 plants they do not matter, so simplicity wins.

Rehashing and game lag

The game's table rehashes when (entries + 1) / buckets would exceed 0.75 and doubles the bucket count. I traced the thresholds by hand from the code (8, 16, 32, 64, then 128 buckets); I did not run this trace.

How this prevents lag as the garden grows:

- **Chains stay short.** Because the table doubles before the load factor passes 0.75, the average chain stays under one entry, so lookups remain close to O(1) however large the garden gets.
- **Doubling keeps inserts cheap on average.** Each rehash costs O(n), but doubling makes the cost O(1) per insert amortize. Growing by a fixed number of buckets would not.
- **Rehash while loading, not mid-day.** When the player moves to a bigger farm the game rebuilds its state, so I would size the registry once at that moment for all n x n plots (for example 81 plants gives 128 buckets). The expensive step then happens during the farm change instead of in the middle of a 30-second day.
- **Honest scale note.** At 25 to 81 plants a rehash takes microseconds and would not be noticeable. The doubling rule is what protects the game if the grid or the number of plants grows much larger.